

GALILEO Graph Databases Group

Implementation of GALOIS Features

Francesco Pivotto 2158296

Giorgia Amato 2159999

Alessio Demo 2142885

December 23, 2025

Contents

1	Iterative Baseline Extension	3
1.1	Motivation and Objective	3
1.2	Design Principles and Architectural Integration	3
1.3	Iterative Execution Workflow	3
1.4	Baseline-Specific Considerations	4
1.5	Outcome and Observations	4
2	Evaluation of the Iterative Baselines	4
2.1	Iterative Execution Results	4
2.2	Comparison with Non-Iterative Baselines	5
3	GALOIS Goal and System Architecture	6
3.1	Orchestration and Schema Management	6
3.2	The Execution Engine	6
4	Logical Optimization (Query Planning)	8
4.1	Predicate Push-Down Strategy	8
4.2	Join Handling Details	8
5	Physical Optimization: Execution Strategies	9
5.1	Table-Scan Algorithm Technical Specification	9
5.1.1	Description	9
5.1.2	Inputs	9
5.2	Key-Scan Algorithm Technical Specification	11
5.2.1	Description	11
5.2.2	Inputs	11
5.3	Adaptive Strategy Selection and Confidence Estimation	13
6	Execution Cycle & Memory Handling	14
6.1	The GaloisMemory Management Class	14
6.1.1	Purpose and Architecture	14

6.1.2	Internal State Variables	14
6.1.3	Core Methods and Functionality	14
6.2	The Iterative Execution Loop and Termination Criteria	15
6.2.1	Loop Mechanism: Pagination & History Injection	15
6.3	Termination Criteria (Convergence Conditions)	15
7	Post-Processing	17
7.1	Robust JSON Repairing	17
7.2	Data Aggregation and Virtualization	17
7.3	Heuristic Type Inference and Sanitization	17
7.4	Deterministic Query Execution	17
8	Numerical Results	19
8.1	GALOIS Variant Comparison	19
9	Experiments	21
9.1	Experiment 1: Overall Evaluation	21
9.2	Experiment 2: Logical and Physical Optimization Impact	21
9.3	Experiment 3: Impact of LLM Parameters	22
9.3.1	Discussion of Results	23
9.3.2	Confidence Estimation Analysis	23
9.4	Experiment 4: Impact of Retrieved Values	24
9.5	Experiment 5: Impact of SQL Query Complexity	24
9.6	Experiment 6: Evolution of Prompt Size	26
9.7	Experiment 7: Threshold Setup	27
9.7.1	Analysis of Calibration Results	27
9.8	Experiment 8: Errors w.r.t. the optimal plan	28
10	Implementation Challenges & Future Work	30
10.1	Performance Deficiencies of the Key-Scan Operator	30
10.2	Database Resource Management for DuckDB	30
10.3	Addressing Sub-Optimal Prompt Structure	30
11	Contributions	32

1 Iterative Baseline Extension

1.1 Motivation and Objective

Building on the unified execution pipeline introduced in previous phases, this task extends both the SQL and NL baselines with a controlled iterative retrieval mechanism. The goal is to improve result completeness by allowing the LLM to generate multiple partial result sets across successive calls, while preserving full compatibility with the project’s standardized JSON output schema.

This mechanism addresses cases where a single LLM invocation fails to recover the entire expected answer set due to truncation, prompt limitations, or generation variability. Iteration provides a systematic way to accumulate non-overlapping tuples across rounds.

1.2 Design Principles and Architectural Integration

The iterative enhancement was introduced without modifying the core architecture. A configurable parameter `max_iter` (defined in `config.yaml`) controls the behavior and ensures consistent execution across providers. The integration adheres to the following principles:

Provider-Agnostic Execution Iteration is implemented at the baseline level (SQL and NL), leaving LLM instantiation and LCEL chain construction unchanged. This guarantees full compatibility with any provider supported through the Adapter and Factory patterns.

Minimal Invasiveness No modifications were made to the LCEL chain, output parser, or prompt template. Iteration is strictly an additional layer wrapped around the existing call-parse-store cycle.

Deduplicated Accumulation Each iteration contributes new tuples based on strict key-wise comparison. A local accumulator collects all rows returned in previous rounds, preventing redundancy and guaranteeing stable output.

Early Stopping Criterion To avoid unnecessary LLM calls, the system terminates early if an iteration yields zero new rows. This rule applies identically to SQL and NL baselines.

1.3 Iterative Execution Workflow

For each query (SQL baseline) or natural language prompt (NL baseline), the extended workflow proceeds as follows:

1. Initialization The system creates an accumulator for unique tuples and loads the `max_iter` parameter.

2. Repeated Invocation At each iteration, the baseline executes a sequence of operations: construction of the LCEL chain with the existing prompt template, LLM invocation using the configured provider, and response parsing via the Pydantic output parser.

3. Deduplication and Union New tuples are compared to the accumulator using canonical JSON formatting, retaining only unseen tuples.

4. Stopping Condition The loop terminates immediately when no new tuples are found in a given iteration.

5. Persistence After early stopping or reaching `max_iter`, the system serializes the union of all retrieved tuples, the total elapsed time, and the aggregated token usage.

1.4 Baseline-Specific Considerations

SQL Baseline Iteration compensates for incomplete tuple enumeration in a single pass. Because SQL inputs are fully specified, repeated calls primarily improve recall.

NL Baseline Iteration mitigates incomplete enumerations and semantic drift in free-form queries. Early stopping is essential to prevent hallucinated or reformulated rows once the LLM stabilizes on a fixed subset.

1.5 Outcome and Observations

Controlled iteration increases tuple coverage while remaining fully aligned with the evaluation framework. The design preserves modularity, provider-agnosticism, and the standardized JSON format established in previous phases.

The mechanism also maintains conceptual continuity with iterative operators in the GALOIS framework (e.g., `Key-Scan`), providing a foundation for more advanced retrieval strategies in future work.

2 Evaluation of the Iterative Baselines

We evaluated the iterative SQL and NL baselines on the full benchmark suite (FLIGHT-2, FLIGHT-4, GEO, MOVIES, PRESIDENTS, WORLD).

2.1 Iterative Execution Results

First of all, concerning the `llama-3.3-70b-instruct` model, the following tables highlight the differences in terms of metrics between results presented in the original GALOIS paper and our implementation with the iterative extension.

Metric	PAPER	Ours	$\Delta(\text{PAPER} - \text{Ours})$
F1-Cell	0.431	0.403	0.028
Cardinality	0.659	0.465	0.194
Tuple Constr.	0.351	0.197	0.154
AVG-Score	0.481	0.355	0.126
#Tokens (M)	0.330	0.008	0.322
Avg Time	61.400	5.263	56.137

Table 1: Results for `sql_baseline`

Metric	PAPER	Ours	$\Delta(\text{PAPER} - \text{Ours})$
F1-Cell	0.237	0.271	-0.034
Cardinality	0.462	0.500	-0.038
Tuple Constr.	0.065	0.091	-0.026
AVG-Score	0.254	0.287	-0.033
#Tokens (M)	0.830	0.005	0.825
Avg Time	120.000	2.206	117.794

Table 2: Results for `nl_baseline`

Maintaining consistency with the previous report, the following table shows the results obtained with the iterative extension for both baselines using the `gemini-2.5-flash` model:

Metric	NL	SQL
F1-Cell	0.425	0.456
Cardinality	0.696	0.607
Tuple Constr.	0.262	0.245
#Tokens (M)	0.028	0.067
Avg Time (s)	5.035	9.681
AVG-Score	0.461	0.436

Table 3: Iterative Results for `gemini-2.5-flash`

The iterative execution impacts the behavior of both baselines in distinct ways. In the SQL setting, the model benefits from task repetition, often recovering additional valid tuples that were missed in the first pass. This results in higher tuple-level completeness and, in many cases, improvements in the F1-CELL and CARDINALITY metrics. For NL queries, which are inherently more open-ended, the iterative mechanism still enhances tuple coverage, though with a less uniform effect across datasets.

2.2 Comparison with Non-Iterative Baselines

To contextualize the effect of iteration, we compare the results above with the single-pass baselines. Overall, the iterative variant provides measurable gains in tuple completeness and global AVG-SCORE, especially in datasets where the model tended to omit large fractions of the answer in a single call. As expected, the iterative mechanism also increases the total number of generated tokens and the average execution time. However, these costs remain moderate and proportional to the configured value of `max_iter`.

In conclusion, iterative execution improves robustness and recall, providing a more stable foundation for evaluating LLM behavior under both SQL and NL interfaces.

3 GALOIS Goal and System Architecture

Large Language Models (LLMs) can answer declarative queries expressed in Natural Language (NL) or SQL, but their one-shot responses frequently fail to recover complete and factually consistent tuples when queries involve filtering, joins, or aggregations. As highlighted in the GALOIS paper, direct prompting exhibits limited recall and incomplete enumeration, making LLMs unreliable as standalone query processors.

The GALOIS framework mitigates these limitations by treating the LLM as a latent knowledge store and decomposing execution into logical and physical operators, in analogy with traditional DBMSs. Rather than issuing a single prompt, GALOIS executes structured scan and filtering strategies (such as `Table-Scan` and `Key-Scan`) together with multiple logical planning variants (`push-all`, `push-selective`, `push-wo`, and the optimized `GALOISF`). This operator-based design improves tuple completeness and stabilizes execution across heterogeneous query types.

Our implementation orchestrates the interaction between a declarative SQL interface and the probabilistic nature of LLMs through four distinct stages: parsing, logical planning, physical execution, and local post-processing. This section details the core components implemented in the `Galois` orchestration class.

3.1 Orchestration and Schema Management

The entry point of the system is the `Galois` class, which initializes the workflow by parsing the input SQL query into an Abstract Syntax Tree (AST) using the `sql_query_parser` module. Unlike standard text-to-SQL approaches, GALOIS does not immediately prompt the LLM with the raw query; instead, it inspects the database structure using the `GaloisSchemaManager`. This component connects to a read-only `DuckDB` instance to retrieve accurate metadata (column names, primary keys, and data types). This schema awareness allows the system to construct semantically valid JSON schemas for the prompts, ensuring that the LLM’s output adheres to the strict typing required for the subsequent relational operations.

Data-Driven Schema Enrichment: The `GaloisSchemaManager` has been enhanced to support few-shot context injection. For textual columns (`VARCHAR` or `TEXT`) that are not primary keys, the manager queries the underlying `DuckDB` instance to retrieve a small sample of actual values (e.g., 3 examples). These examples are injected directly into the JSON schema description passed to the LLM (e.g., under the key `"description"`), providing the model with immediate context about data formatting and distribution.

3.2 The Execution Engine

The retrieval logic is encapsulated within the `GaloisExecutor` class. This component abstracts the complexity of the LLM interaction (via `LangChain`) and implements the iterative retrieval loops.

State Management and Deduplication: To handle the context window limitations of LLMs, the executor utilizes a custom `GaloisMemory` class. This memory component maintains a history of retrieved tuples and implements a strict deduplication mechanism based on primary key hashing using the internal helper `_get_key_values`. This ensures

that subsequent iterations (up to a configurable `max_iter`) prompt the model to generate only novel data.

Iterative Scan Operators: The executor supports two physical operators. The **Table-Scan** iteratively requests complete tuples (keys and attributes) until convergence or iteration limits are reached. Alternatively, the **Key-Scan** decomposes retrieval into two phases: first, it iterates to collect all unique primary keys; once the keys are consolidated, it issues a final "Tuple Request" to retrieve non-key attributes for the collected identifiers.

4 Logical Optimization (Query Planning)

The Logical Optimization phase determines what information needs to be extracted from the LLM and which filtering conditions should be "pushed down" to the generative model versus applied locally. This decision-making process is critical to balancing the trade-off between retrieval precision and the risk of hallucination.

4.1 Predicate Push-Down Strategy

The system implements a flexible planning mechanism capable of generating different execution plans based on the configured optimization level. The planner analyzes the **WHERE** clause of the parsed SQL query and partitions the conditions into two sets: **conditions_to_push** (integrated into the prompt) and **residual_conditions** (deferred to the post-processor). The framework supports four distinct logical variants.

No-Push (GALOIS_{WO}) This strategy treats the LLM purely as a data source. No filtering conditions are included in the prompt; the system retrieves the entire extension of the table (or its keys) using the **Key-Scan** operator and performs all filtering locally. This approach minimizes the reasoning burden on the model but maximizes token consumption.

Push-All (GALOIS_A) This variant maximizes delegation by translating all SQL predicates into natural language constraints within the prompt. It assumes high model capability in understanding and applying complex logic. While potentially the most efficient in terms of token usage, it carries a higher risk of the model hallucinating or missing tuples that satisfy complex boolean conditions.

Push-Selective (GALOIS_S) A heuristic approach that identifies and pushes only the most selective condition. The system uses the LLM’s confidence self-assessment as a proxy to determine which conditions are “safe” or “selective” enough to be handled directly by the generative model. The aim is to reduce the search space without overwhelming the model’s context window with excessive constraints.

Push-Confident (GALOIS_F) An adaptive strategy that utilizes a **ConfidenceEstimator**. The system queries the LLM to self-evaluate its confidence in correctly assessing specific predicates (returning a "HIGH" or "LOW" score). Only conditions with high confidence scores are pushed down, while ambiguous ones are retained for local execution. Furthermore, the LLM performs an estimation process over the physical optimization strategy for the entire query; details on this mechanism are discussed in Section 5.3.

4.2 Join Handling Details

For multi-table queries, the logical planner decomposes the join operation into independent scan tasks. The system identifies all tables involved in the query (via the **FROM** and **JOIN** clauses) and generates a sub-plan for each. The planner ensures that alias references in the SQL query are correctly mapped to their respective table scans, allowing the **GaloisExecutor** to fetch data for each relation independently before they are rejoined in the Post-Processing phase.

5 Physical Optimization: Execution Strategies

While Logical Optimization defines the declarative scope of the data retrieval task, Physical Optimization is responsible for specifying the concrete, algorithmic execution strategy. The proposed system implements a set of iterative data extraction algorithms designed to mitigate the inherent context window limitations of Large Language Models (LLMs) and ensure comprehensive, de-duplicated data acquisition.

5.1 Table-Scan Algorithm Technical Specification

5.1.1 Description

The **Table-Scan** algorithm is a single-phase, iterative extraction technique that leverages an LLM to retrieve a complete collection of unique, full-schema records (tuples) satisfying the input query Q . This operator is designed to maximize coverage by focusing on the generation and accumulation of entire records in each iteration step, contrasting with multi-phase strategies.

The execution process begins with an iterative tuple collection loop (Algorithm 1, Lines 4–19), which repeatedly invokes the LLM chain to discover and accumulate novel, unique tuples that conform to the constraints of Q . The initial invocation ($i = 0$) utilizes a base input prompt derived directly from the query. For subsequent iterations ($i > 0$), the algorithm dynamically modifies the input request by injecting the historical context ($\mathcal{H}$) of previously accumulated tuples. This mechanism guides the LLM to seek novel, uncollected records, thereby enforcing deduplication and maximizing the result set. The process is bounded by a maximum number of iterations (I_{max}), but it also features an early termination condition: if the post-response parsing step confirms the absence of new unique tuples ($\mathcal{R}_{new} = \emptyset$), the loop is prematurely exited to handle convergence efficiently.

Upon termination of the loop, the finalization phase (Lines 22–24) retrieves the consolidated set of unique tuples stored in the internal memory structure ($\mathcal{M}$), designating it as the final result set $\mathcal{R}_{final}$. The state of the internal memory and history is subsequently reset to ensure clean execution for future queries.

5.1.2 Inputs

The algorithm requires the following parameters for execution:

Parameter	Description
Q	The declarative data retrieval statement (e.g., an SQL-like query).
$\mathcal{P}_{pushdown}$	An optional set of WHERE clause predicates pushed down to refine the extraction scope.
I_{max}	An optional integer parameter specifying the maximum number of LLM invocations.

Table 4: Input Parameters for Table-Scan

Algorithm 1 Table-Scan Algorithm

```
1: function TABLE_SCAN( $Q, \mathcal{P}_{pushdown}, I_{max}$ )
2:    $\mathcal{I} \leftarrow \text{BUILDINITIALREQUEST}(Q)$ 
3:    $\mathcal{C} \leftarrow \text{INITIALIZELLMINTERACTIONCHAIN}$ 
4:    $i \leftarrow 0$ 
5:    $\mathcal{H} \leftarrow \emptyset$ 
6:
7:   while  $i < I_{max}$  do
8:     if  $i = 0$  then
9:        $\mathcal{R}_{raw} \leftarrow \mathcal{C}.\text{INVOKE}(\mathcal{I})$ 
10:    else
11:       $\mathcal{I} \leftarrow \text{UPDATEREQUESTWITHHISTORY}(\mathcal{H})$ 
12:       $\mathcal{R}_{raw} \leftarrow \mathcal{C}.\text{INVOKE}(\mathcal{I})$ 
13:
14:       $\mathcal{R}_{parsed} \leftarrow \text{PARSELMLRESPONSE}(\mathcal{R}_{raw})$ 
15:       $\mathcal{R}_{new} \leftarrow \text{IDENTIFYNEWUNIQUETUPLES}(\mathcal{R}_{parsed}, \mathcal{H})$ 
16:      if  $\mathcal{R}_{new} = \emptyset$  then
17:        break
18:
19:       $\text{ADDTOMEMORYANDHISTORY}(\mathcal{R}_{new}, \mathcal{M}, \mathcal{H})$ 
20:       $i \leftarrow i + 1$ 
21:
22:   $\mathcal{R}_{final} \leftarrow \text{RETRIEVEALLTUPLES}(\mathcal{M})$ 
23:   $\text{CLEARSTATE}(\mathcal{M}, \mathcal{H})$ 
24:
25:  return  $\mathcal{R}_{final}$ 
```

5.2 Key-Scan Algorithm Technical Specification

5.2.1 Description

The **Key-Scan** algorithm constitutes a specialized, two-phase iterative data extraction strategy. It employs an LLM to retrieve the required record set by prioritizing the discovery and consolidation of unique primary or candidate key values before performing a final request for associated, non-key attributes. This two-phase approach inherently ensures record uniqueness and improves token efficiency by minimizing the size of the history context injected into the prompt.

The first phase, Iterative Key Collection (Algorithm 2, Lines 4–19), iteratively invokes the LLM chain to discover and accumulate a complete, unique set of key values ($\mathcal{K}$) relevant to the query Q . The initial iteration uses a prompt formulated to extract only the key attributes. Subsequent iterations dynamically modify the input request by embedding a history ($\mathcal{H}$) of previously collected key values, steering the LLM’s generation toward novel, uncollected records. This loop is governed by a maximum iteration limit (I_{max}) and a convergence condition: it terminates early if the parsing step confirms that no new unique key values ($\mathcal{K}_{new} = \emptyset$) were returned.

Upon convergence of the key collection, the algorithm proceeds to the Final Attribute Retrieval phase (Lines 22–26). The consolidated set of all unique keys ($\mathcal{K}_{all}$) is compiled, and a single, dedicated final request (the *Tail Request*) is issued to the LLM. This request efficiently fetches all missing non-key attributes ($\mathcal{A}$) corresponding to the collected keys. The function concludes by performing a relational join operation between the collected keys and the attributes from the final response to construct the final set of complete, de-duplicated records ($\mathcal{R}_{final}$).

5.2.2 Inputs

The algorithm accepts the following parameters:

Parameter	Description
Q	The SQL-like data retrieval statement.
$\mathcal{P}_{pushdown}$	A list of WHERE clause predicates for extraction refinement.
I_{max}	An optional integer parameter bounding the iterative key collection phase.

Table 5: Input Parameters for Key-Scan

Algorithm 2 Key-Scan Algorithm

```
1: function KEY_SCAN( $Q, \mathcal{P}_{pushdown}, I_{max}$ )
2:    $\mathcal{I} \leftarrow \text{BUILDINITIALREQUEST}(Q)$ 
3:    $\mathcal{C} \leftarrow \text{INITIALIZELLMINTERACTIONCHAIN}$ 
4:    $i \leftarrow 0$ 
5:    $\mathcal{H} \leftarrow \emptyset$ 
6:
7:   while  $i < I_{max}$  do
8:     if  $i = 0$  then
9:        $\mathcal{R}_{raw} \leftarrow \mathcal{C}.\text{INVOKE}(\mathcal{I})$ 
10:    else
11:       $\mathcal{I} \leftarrow \text{UPDATEREQUESTWITHHISTORY}(\mathcal{H})$ 
12:       $\mathcal{R}_{raw} \leftarrow \mathcal{C}.\text{INVOKE}(\mathcal{I})$ 
13:
14:       $\mathcal{K}_{parsed} \leftarrow \text{PARSELLMRESPONSE}(\mathcal{R}_{raw})$ 
15:       $\mathcal{K}_{new} \leftarrow \text{IDENTIFYNEWUNIQUEKEYS}(\mathcal{K}_{parsed}, \mathcal{H})$ 
16:      if  $\mathcal{K}_{new} = \emptyset$  then
17:        break
18:
19:       $\text{SAVEKEYVALUESANDUPDATEHISTORY}(\mathcal{K}_{new}, \mathcal{H})$ 
20:       $i \leftarrow i + 1$ 
21:
22:    $\mathcal{K}_{all} \leftarrow \text{RETRIEVEALLCOLLECTEDUNIQUEKEYS}(\mathcal{H})$ 
23:    $\mathcal{I}_{final} \leftarrow \text{BUILDFINALATTRIBUTEREQUEST}(\mathcal{K}_{all})$ 
24:    $\mathcal{R}_{final\_raw} \leftarrow \mathcal{C}.\text{INVOKE}(\mathcal{I}_{final})$ 
25:    $\mathcal{A}_{parsed} \leftarrow \text{PARSEFINALRESPONSE}(\mathcal{R}_{final\_raw})$ 
26:    $\mathcal{R}_{final} \leftarrow \text{RELATIONALJOIN}(\mathcal{K}_{all}, \mathcal{A}_{parsed})$ 
27:    $\text{CLEARSTATE}(\mathcal{H})$ 
28:
29:   return  $\mathcal{R}_{final}$ 
```

5.3 Adaptive Strategy Selection and Confidence Estimation

The framework incorporates an adaptive execution layer that dynamically mediates the selection between the *Table-Scan* and *Key-Scan* operators. This decision is parameterized by a Confidence Estimator, which quantitatively assesses the expected factual data retrieval reliability based on the schema complexity of the query (Q) and the LLM’s raw confidence score.

The confidence estimation model incorporates a decay factor sensitive to the number of projected columns. The final confidence score C_{final} is calculated from the LLM’s intrinsic raw confidence (C_{raw}) by applying an exponential decay based on N_{cols} , representing the cardinality of attributes in the **SELECT** clause:

$$C_{final} = (C_{raw})^{N_{cols}}$$

This formulation rigorously models the non-linear increase in difficulty for the LLM to generate a coherent, fully populated tuple set as the schema width grows.

Execution Strategy Selection Heuristic The selection logic compares the derived score against a predefined operational threshold (τ). If C_{final} exceeds τ , the system prioritizes the *Key-Scan* approach. This two-phase method aligns with a Chain-of-Thought reasoning process, which is generally associated with higher accuracy and factuality when the model exhibits sufficient confidence. Conversely, if C_{final} falls below τ , or if the schema is judged to be excessively wide ($N_{cols} \gg 1$), the framework defaults to the *Table-Scan* operator. *Table-Scan* is considered the more robust strategy when factual confidence is low, as it reduces the reasoning complexity of the initial extraction target.

This adaptive mechanism is fundamental for optimizing the execution of the full GALOIS_F logical plan by selecting the most appropriate physical operator based on a data-driven confidence metric.

6 Execution Cycle & Memory Handling

The interaction between the deterministic orchestration layer and the generative model is governed by a stateful execution cycle. Unlike standard single-turn LLM interactions, the Galois framework maintains a persistence layer across iterations to maximize recall and prevent redundancy. This logic is encapsulated within the `GaloisExecutor` class and its specialized internal memory component, `GaloisMemory`.

6.1 The GaloisMemory Management Class

6.1.1 Purpose and Architecture

The `GaloisMemory` class serves as the dedicated state management component for the physical execution strategies (`Table-Scan` and `Key-Scan`). Inheriting from `BaseMemory` and `BaseModel`, it is architected to fulfill three primary responsibilities: maintaining an accumulated history of unique records and their logprobs extracted across multiple iterative steps, enforcing strict deduplication to preserve data integrity, and formatting these accumulated records into a structured context ($\mathcal{H}$) for injection into subsequent LLM prompts. By managing this persistent state, the class ensures that the model is continuously guided toward novel data discovery while preventing redundant processing.

6.1.2 Internal State Variables

The memory state is maintained by a set of core attributes defined as Pydantic fields. These variables track the data accumulation, deduplication status, and execution metrics:

Attribute & Type	Description
<code>memory</code> <code>List[Dict[str, Any]]</code>	The primary storage structure containing the complete collection of unique records (tuples) retrieved thus far.
<code>key_values</code> <code>set</code>	A hash set storing unique key value combinations (as Python tuples).
<code>tokens</code> <code>int</code>	An aggregate counter tracking the total number of tokens processed by the LLM across all invocations.
<code>time</code> <code>float</code>	An aggregate measure of the total processing time (latency) associated with the LLM interactions.
<code>logprobs</code> <code>List[float]</code>	A list storing the average log-probabilities for each generated record, used to estimate confidence.

Table 6: Internal State Attributes of `GaloisMemory`

6.1.3 Core Methods and Functionality

The functional logic of the memory component is implemented through four distinct methods that handle the lifecycle of data persistence.

The `save_context(inputs, outputs)` method acts as the central mechanism for processing new records generated by an LLM invocation. It iterates through the provided

response records and uses the internal helper `_get_key_values(record, keys)` to extract composite key tuples. These keys are checked against the `self.key_values` set; if a key is absent, the record is appended to `self.memory` and the key is registered.

Simultaneously, the method preserves the pre-calculated average log-probabilities associated with each tuple into the `self.logprobs` structure, maintaining a strict one-to-one correspondence between the stored records and their confidence scores for subsequent quality filtering.

Crucially, this method implements the convergence logic: if the size of the key set remains unchanged after processing, it raises a `NoNewTuplesFound` exception to signal the termination of the iterative process. Additionally, it aggregates execution metrics (`tokens` and `time`) from the output.

To support the iterative prompting cycle, the `load_memory_variables(inputs)` method prepares the accumulated records for injection into the next prompt. It serializes the current `self.memory` list into a compact JSON string, returning it under the key `"history"`.

Finally, the `clear()` method provides a reset mechanism, emptying the storage structures and zeroing out metrics to ensure a clean state for subsequent query executions.

6.2 The Iterative Execution Loop and Termination Criteria

Both the **Table-Scan** and **Key-Scan** physical operators rely on a fundamental Iterative Execution Loop (lines 4-19 in Algorithms 1 and 2) to overcome the LLM context window limitations and maximize data extraction coverage. The core mechanism involves repeatedly querying the LLM and dynamically updating the prompt with historical data.

6.2.1 Loop Mechanism: Pagination & History Injection

In both strategies, the iterative process is driven by the dynamic insertion of previously acquired data into the subsequent prompt. This mechanism ensures two critical functions: **Deduplication**, where the LLM is implicitly guided to avoid regenerating records already present in the injected history ($\mathcal{H}$), and **Novelty Search**, where the context directs the LLM to explore uncollected portions of the solution space to maximize the cardinality of the final result set ($\mathcal{R}_{final}$).

To enhance retrieval efficiency and prevent the LLM from wandering randomly, the iterative mechanism implements a **value-based pagination** strategy. In addition to the history, the system identifies the last retrieved value (`last_val`) from the memory. The prompt logic explicitly instructs the LLM to generate unique tuples that strictly follow `last_val` (alphabetically or numerically). This transforms the extraction into a pseudo-sequential scan, significantly improving coverage compared to simple history injection. The primary distinction between the operators lies in the content of the history: **Table-Scan** injects a history of complete tuples, while **Key-Scan** injects only the unique key values ($\mathcal{K}$) collected so far.

6.3 Termination Criteria (Convergence Conditions)

The iterative loop is governed by two distinct termination criteria, ensuring both safety against infinite execution and functional completeness. The system employs a `break`

statement within the execution loop immediately when either condition is met.

The first criterion is the **Maximum Iteration Limit** (I_{max}). This condition is triggered when the loop counter (i) reaches a maximum permissible value, which is configurable via the system’s YAML parameter file. The primary purpose of this limit is to serve as a resource fail-safe, strictly bounding token consumption and total execution latency. It prevents the system from entering infinite loops or wasting computational resources in scenarios where the LLM might continue to generate redundant or low-quality data indefinitely.

The second criterion is **Data Convergence**, which serves as the primary functional exit condition. This state is reached when the system determines that an LLM invocation has yielded **no new unique records** (for Table-Scan) or **no new unique keys** (for Key-Scan). This condition is explicitly detected by the `save_context` function within the `GaloisMemory` class, which monitors the growth of the internal key set. When the set size remains static after a response, it signals that the LLM has likely exhausted its capacity to discover novel data relevant to the query, allowing the loop to terminate prematurely to maximize efficiency.

The system employs a `break` statement within the execution loop immediately when convergence is detected, effectively overriding the I_{max} constraint if full coverage is achieved earlier.

7 Post-Processing

The Post-Processing module serves as the deterministic convergence point of the architecture, addressing the inherent impedance mismatch between the probabilistic output of the Large Language Model and the strict requirements of the SQL standard. This phase transforms the raw tuple sets retrieved by the Executor into a consistent relational state, enabling the execution of complex logical operations that cannot be reliably offloaded to the generative model.

7.1 Robust JSON Repairing

To address the issue of invalid JSON syntax produced by LLMs (e.g., trailing commas, missing null values, or markdown code blocks), we implemented a deterministic sanitization routine (`repair_json_content`) within the Executor. Before parsing, the raw response creates a sanitized string using regular expressions to strip Markdown tags and fix common syntax errors (such as converting empty assignments `key:` , to `key: null,`). This preprocessing step significantly reduced parsing failures.

7.2 Data Aggregation and Virtualization

Upon the completion of the physical scan phase (whether *Table-Scan* or *Key-Scan*), the retrieved tuples are aggregated into a local staging structure, referred to as the **Data Lake**, which maps table names to lists of dictionaries. To bridge the gap to a relational environment, the system instantiates an ephemeral, in-memory **DuckDB** connection. The aggregated data is converted into **Pandas** DataFrames and registered as virtual views within this database instance, effectively creating a transient relational schema populated by the LLM-generated data.

7.3 Heuristic Type Inference and Sanitization

The system implements a strict type inference policy to preserve data integrity. The `perform_local_join_and_query` method employs specific regular expressions to identify and normalize various string representations of null produced by LLMs (e.g., `'nan'`, `'null'`, `'n/a'`) into native Python `None` objects. Furthermore, a strict casting policy is applied via **Pandas**: columns are converted to numeric types only if the conversion preserves data integrity (checking the ratio of numeric vs. non-null values); otherwise, they revert to object types to avoid data loss during the virtualization in **DuckDB**. This zero-tolerance approach ensures that alphanumeric identifiers or mixed-format codes are not erroneously coerced into null values.

7.4 Deterministic Query Execution

Once the data is virtualized and typed, the system executes the logical query plan locally. This step involves running the original SQL query (sanitized of specific schema prefixes like `.target`) against the virtual tables. This local execution performs two vital functions:

Join Reconstruction It re-establishes the relational links between disparate datasets that were retrieved via independent scan operations, ensuring the correct implementation of **INNER** or **LEFT** joins defined in the original AST.

Residual Filtering It applies the `residual_conditions` predicates identified during the Logical Optimization phase as too complex or numerically precise for the LLM (e.g., strict inequalities like `length_in_km > 750`). This ensures that the final result set adheres strictly to the logical constraints of the user’s query, filtering out any hallucinations or imprecise data artifacts introduced by the model.

8 Numerical Results

In this section, we present a comparison between the metrics reported in the original GALOIS paper and those obtained from our implementation.

8.1 GALOIS Variant Comparison

Table 7: GALOIS_{WO} Comparison

Metric	PAPER	Ours	Δ
F1-Cell	0.518	0.157	0.361
Cardinality	0.691	0.357	0.334
Tuple Constr.	0.389	0.060	0.329
AVG	0.531	0.191	0.340
#Tokens (M)	19.71	0.16	19.71
Avg Time	1460.0	43.1	1416.9

Table 8: GALOIS_S Comparison

Metric	PAPER	Ours	Δ
F1-Cell	0.480	0.434	0.046
Cardinality	0.655	0.667	-0.012
Tuple Constr.	0.365	0.297	0.068
AVG	0.500	0.466	0.034
#Tokens (M)	0.96	0.09	0.87
Avg Time	130.0	18.0	112.0

Table 9: GALOIS_A Comparison

Metric	PAPER	Ours	Δ
F1-Cell	0.543	0.417	0.126
Cardinality	0.799	0.614	0.185
Tuple Constr.	0.448	0.279	0.169
AVG	0.592	0.436	0.156
#Tokens (M)	0.95	0.08	0.87
Avg Time	120.5	24.5	96.0

Table 10: GALOIS_F Comparison

Metric	PAPER	Ours	Δ
F1-Cell	0.563	0.319	0.244
Cardinality	0.835	0.528	0.307
Tuple Constr.	0.464	0.166	0.298
AVG	0.622	0.338	0.284
#Tokens (M)	1.72	0.14	1.58
Avg Time	47.4	26.2	21.2

As illustrated in the tables above, the most significant performance divergence is observed in the GALOIS_{WO} version. This shortfall can be attributed to three primary factors: the probabilistic nature of "Context-Free" retrieval, the loss of semantic cohesion in the Key-Scan operator, and the depth of iteration in simulated table scans.

Context-Free Retrieval and Popularity Bias The primary cause of the low recall in GALOIS_{WO} is the absence of predicate pushdown. By design, this variant forces the logical plan to strip all **WHERE** clauses, presenting the LLM with a generic query (e.g., "List all presidents" instead of "List presidents of Venezuela"). Unlike a deterministic database storage engine that scans all records faithfully, an LLM behaves as a probabilistic engine heavily influenced by popularity bias. Without the "guidance" of specific filters in the prompt, the model tends to generate only general or highly popular entities. Since the Post-Processing module can only filter data that has been successfully retrieved, the lack of relevant entries renders the local filtering phase ineffective.

Semantic Decohesion in the Key-Scan Operator Our implementation highlights a structural weakness in the Key-Scan physical operator, specifically in how it decouples the retrieval of keys (Phase 1) from their attributes (Phase 2). In a standard **Table-Scan** (used in GALOIS_A and GALOIS_S), the co-occurrence of related tokens in the generation window (e.g., generating "Venezuela" immediately next to a candidate name) helps the LLM's self-attention mechanism maintain semantic consistency. By isolating the key

generation, the **Key-Scan** loses this context. Furthermore, in a "No-Push" scenario, the first phase often wastes significant token budget generating irrelevant keys, which are then discarded during post-processing, drastically reducing the system's overall efficiency.

The Trade-off Between Execution Time and Scan Depth A critical divergence from the reference paper is observed in the average execution time. This discrepancy indicates that our implementation imposes a stricter limit on iterations (`max_iter` defaults to 4 in `executor.py`). To successfully simulate a "Full Table Scan" on an LLM—essentially brute-forcing the model to output its entire knowledge base for a given entity—a system must iterate significantly longer to bypass the initial layer of popular results. Our lower execution time suggests that the retrieval process is terminated prematurely, capturing only the "surface" data. This implies that GALOIS_{WO} is theoretically viable only with massive iteration depth, which may be inefficient for real-world applications due to latency and token cost constraints. Conversely, GALOIS_S and GALOIS_A perform better because the prompt acts as a "soft index," allowing the LLM to jump directly to the relevant data partition without requiring exhaustive iteration.

9 Experiments

9.1 Experiment 1: Overall Evaluation

In this experiment, we obtain structured data from the internal parametric knowledge of the LLM. We evaluate the performance of the variants of GALOIS against the NL, SQL, and GALOIS baselines, using `llama-3.3-70b-instruct`. The datasets used are FLIGHT-2, FLIGHT-4, GEO, MOVIES, PRESIDENTS, and WORLD (excluding PREMIER and FORTUNE). A threshold $\tau = 0.6$ is used for Physical Plan selection.

The results comparison between our implementation and the former GALOIS paper is shown in the following tables:

Table 11: GALOIS Paper Results

Metric	NL	SQL	Galois _{WO}	Galois _S	Galois _A	Galois _F
F1-Cell	0.237	0.431	0.518	0.480	0.543	0.563
Cardinality	0.462	0.659	0.691	0.655	0.799	0.835
T. Constr.	0.065	0.351	0.389	0.365	0.448	0.464
Avg-Score	0.254	0.481	0.531	0.500	0.592	0.622
#Tokens (M)	0.83	0.33	19.71	0.96	0.95	1.72
Avg Time	120.0	61.4	1460	130	120.5	47.4

Table 12: Our Implementation Results

Metric	NL	SQL	Galois _{WO}	Galois _S	Galois _A	Galois _F
F1-Cell	0.271	0.403	0.157	0.434	0.417	0.319
Cardinality	0.500	0.465	0.357	0.667	0.614	0.528
T. Constr.	0.091	0.197	0.060	0.297	0.279	0.166
Avg-Score	0.287	0.355	0.191	0.466	0.436	0.338
#Tokens (M)	0.005	0.008	0.163	0.088	0.076	0.142
Avg Time	2.206	5.263	43.103	17.952	24.481	26.196

9.2 Experiment 2: Logical and Physical Optimization Impact

This experiment evaluates the effectiveness of the logical and physical optimization strategies adopted by GALOIS, following the methodology described in Sections 4 and 5 of the original paper. The goal is to isolate and quantify how different execution strategies affect result quality, measured through the **AVG-Score** (average of F1-Cell, Cardinality, and Tuple Constraint). All experiments are conducted on the **GEO** dataset, which queries the internal parametric knowledge of the LLM and includes 32 SQL queries with pre-computed ground truth.

Physical Optimization At the physical level, GALOIS selects how data is retrieved from the LLM. For each query, we compare three execution strategies: **Table-Scan**, **Key-Scan**, **AUTO**

Table 13: Physical optimization results on GEO (AVG-Score and token usage).

Strategy	AVG-Score	Tokens (M)
Table-Scan	0.315	0.030
Key-Scan	0.272	0.033
AUTO	0.274	0.031

The results show that, on GEO, **Table-Scan** achieves the **highest average quality**, suggesting that richer contextual prompts help the LLM retrieve more complete and accurate data. The AUTO strategy closely follows the best fixed plan, confirming that the confidence-based physical planner behaves robustly, although the gain over Table-Scan is limited for this dataset.

Logical Optimization To isolate the impact of logical planning decisions, we fix the physical strategy to **Table-Scan** and evaluate different predicate pushdown strategies:

- **NO-PUSH** strategy pushes no conditions into the LLM scan.
- **PUSH-ALL** (GALOIS_A) pushes all **WHERE** conditions down.
- **PUSH-CONFIDENT** (GALOIS_F) pushes only conditions selected by the confidence estimator.

Only queries with at least two **WHERE** conditions are considered.

Table 14: Logical optimization results on GEO with fixed Table-Scan.

Strategy	AVG-Score	Tokens (M)
NO-PUSH	0.318	0.015
GALOIS A (Push-All)	0.274	0.009
GALOIS F (Confident)	0.397	0.014

The results clearly show that **confidence-based pushdown** (GALOIS_F) significantly outperforms both NO-PUSH and PUSH-ALL strategies. This confirms that pushing all predicates may reduce cost but often degrades result quality due to increased reasoning complexity, while selectively pushing only confident predicates yields a better balance between precision and recall.

9.3 Experiment 3: Impact of LLM Parameters

This section presents a study designed to evaluate how the choice of Large Language Model (LLM) and its parameter size influences the performance of the GALOIS system. We tested the framework using three distinct models: **gpt-4o-mini**, **llama-3.1-8b-instruct**, and **llama-3.3-70b-instruct**.

The primary objective was to assess whether the increased reasoning capabilities and parametric knowledge of larger models are essential for the effective execution of the logical and physical optimizations proposed in GALOIS. Furthermore, we investigated the reliability of the models’ confidence scores (logprobs) as a mechanism for filtering results, aiming to balance precision and recall. Performance is measured using the AVG-SCORE,

comparing the fully optimized system (GALOIS_F) against simpler variants (GALOIS_{WO} , GALOIS_A) and standard baselines (NL and SQL).

Table 15: AVG-Score of different LLMs (PAPER RESULTS)

LLM	NL	SQL	$\mathbf{G}_{WO}$	$\mathbf{G}_A$	$\mathbf{G}_F$
GPT-4o MINI	0.258	0.240	0.456	0.457	0.468
LLaMA 3.1 8B	0.230	0.372	0.375	0.520	0.528
LLaMA 3.1 70B	0.254	0.481	0.531	0.592	0.622

Table 16: AVG-Score of different LLMs (OUR RUNS)

LLM	NL	SQL	$\mathbf{G}_{WO}$	$\mathbf{G}_A$	$\mathbf{G}_F$
GPT-4o MINI	0.073	0.295	0.013	0.298	0.166
LLaMA 3.1 8B	0.268	0.372	0.189	0.376	0.189
LLaMA 3.1 70B	0.287	0.355	0.191	0.436	0.338

9.3.1 Discussion of Results

The comparison highlights critical divergences regarding the efficacy of the optimization pipeline. In the original paper, there is a constant and continuous improvement as model size increases and as optimizations are applied. Specifically, GALOIS_F is always the superior configuration, reaching an AVG-SCORE of 0.622 with `llama-3.1-70b`.

However, our reproduction data suggests a regression in full optimization. Unlike the reference paper, where GALOIS_F provides the best results, our experiments show that GALOIS_F consistently underperforms compared to GALOIS_A . This indicates that the additional logic in the full pipeline—likely the dynamic switching to the *Key-Scan* operator or the confidence-based plan selection—is introducing errors rather than resolving them.

Additionally, the baseline performance of the unoptimized GALOIS_{WO} yields significantly lower scores in our runs compared to the paper. This suggests that the fundamental iterative retrieval mechanism without pushdown conditions is struggling to retrieve relevant tuples in our setup.

9.3.2 Confidence Estimation Analysis

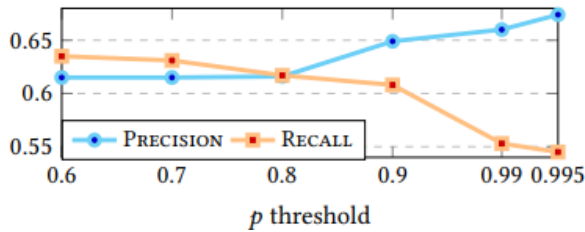


Figure 1: Original Paper Results

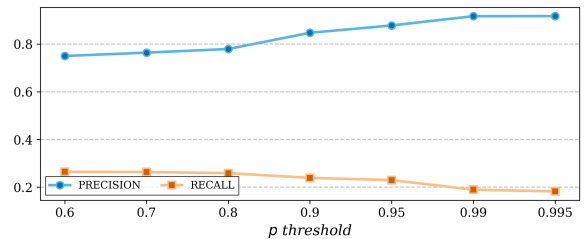


Figure 2: Our Reproduction Results

The results in Figure 2 confirm the expected trade-off: as the threshold p increases from 0.6 to 0.995, **Precision** improves notably in our runs, rising from approximately 0.75 to over 0.90. This demonstrates that the model’s internal confidence is a reliable proxy for the factual correctness of the generated data; when the model is confident, the data is likely correct.

However, the charts also reveal the root cause of the lower overall scores observed in the tables. While the original paper (left) shows **Recall** starting above 0.60, our reproduction (right) shows Recall starting at a very low baseline (approximately 0.26 at $p = 0.6$). Even with lenient filtering, the system retrieves less than 30% of the expected tuples. This low initial recall explains why the AVG-SCORES in our reproduction are capped: even if precision is high, the system simply fails to extract the majority of the required information from the model’s parametric knowledge.

9.4 Experiment 4: Impact of Retrieved Values

We implemented an automated evaluation pipeline focusing on the **PRESIDENTS** domain to quantify how data frequency (rarity) and temporal recency influence LLM performance.

We measure two specific biases: **Rarity**, comparing performance on popular entities (USA) versus rare entities (Venezuela), and **Temporality**, comparing performance across three categories: *All-Time*, *Recent* (late 1900s–today), and *Past* (until late 1900s).

Table 17: Impact of Rarity (Country) on Query Execution Quality (AVG-SCORE).

Category	GALOIS _A	GALOIS _F	GALOIS _S	GALOIS _{WO}	NL	SQL
Presidents-USA	0.570	0.652	0.587	0.089	0.352	0.320
Presidents-Venezuela	0.282	0.212	0.323	0.051	0.091	0.257

Table 18: Impact of Temporality on Query Execution Quality (AVG-SCORE).

Category	GALOIS _A	GALOIS _F	GALOIS _S	GALOIS _{WO}	NL	SQL
Recent	0.532	0.531	0.449	0.095	0.375	0.318
Past	0.240	0.305	0.378	0.000	0.183	0.281
All-Time	0.427	0.420	0.479	0.077	0.160	0.277

The results confirm that query generation quality degrades significantly when dealing with rare or historical data.

9.5 Experiment 5: Impact of SQL Query Complexity

Objective and Methodology The objective of this experiment is to evaluate the robustness and stability of the GALOIS system as the structural and logical complexity of the input SQL query varies. This analysis is crucial given that Large Language Models (LLMs) typically exhibit performance degradation when handling complex logic, such as multiple joins or nested aggregations.

To conduct this analysis, we implemented a static classifier (`classify_query_complexity`) that parses the Abstract Syntax Tree (AST) of each SQL query via `sqlglot`. The classifier assigns each query to one of six mutually exclusive complexity categories, ordered by expected difficulty:

- **SP2 (Selection-Projection Simple)**: Linear queries involving only selection and projection operations with at most 2 conditions in the `WHERE` clause. These represent the simplest base case.
- **SP2> (Selection-Projection Complex)**: Selection/projection queries featuring more than 2 filtering conditions, requiring the LLM to manage a denser logical context.
- **DIST (Distinct)**: Queries including the `DISTINCT` clause, requiring the system to handle explicit result deduplication.
- **AGGR (Aggregation)**: Queries utilizing aggregation functions (`COUNT`, `SUM`, `AVG`, `MAX`, `MIN`), which require arithmetic reasoning or counting capabilities on retrieved tuples.
- **G/O (Group By / Order By)**: Queries imposing sorting or grouping of data, testing the system’s ability to structure output sequentially.
- **JOIN**: Queries involving joins between two or more tables. This is considered the most complex category, as it requires the system to decompose the problem into multiple scans (multi-table retrieval) and reconstruct relationships in local post-processing.

The GALOIS results are compared against the Ground Truth (loaded from pre-calculated JSON files) using three key metrics: F1-Cell Exact, Cardinality, and Tuple Constraint.

Analysis of Results Figure 3 compares the metrics of our run with those from the original GALOIS paper. The results clearly indicate that `JOIN` operations significantly degrade retrieval quality. The primary reason for this drop is the inherent difficulty LLMs face in retrieving tuples that satisfy multi-table constraints—a limitation already observed in the general results discussed in Section 8. While the system performs adequately on linear selection queries (SP2), the decomposition required for joins introduces retrieval gaps that local post-processing cannot always bridge.

Finally, Figure 4 illustrates the token usage of our implementation compared to the original paper, highlighting the cost implications associated with increasing query complexity.

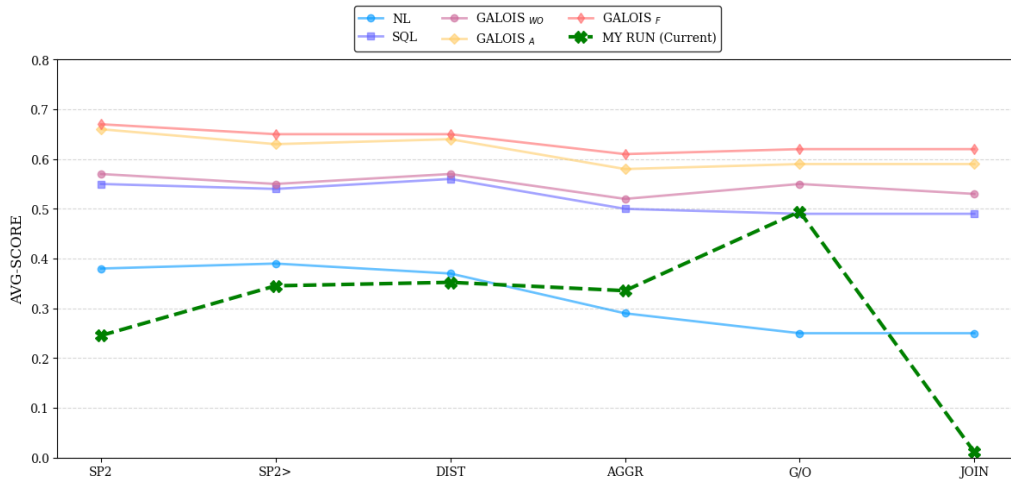


Figure 3: Result quality w.r.t query complexity

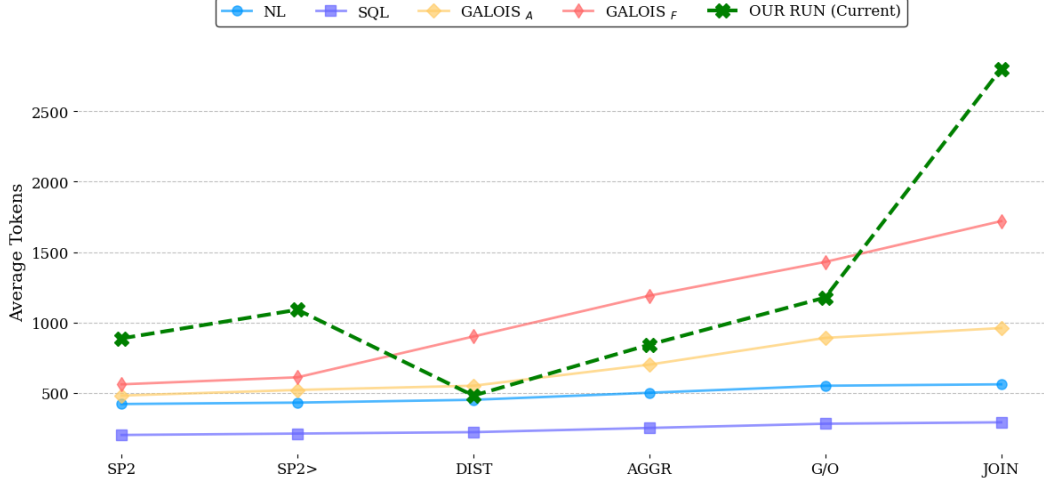


Figure 4: Costs w.r.t query complexity. GALOIS_{WO} (not reported)

9.6 Experiment 6: Evolution of Prompt Size

The goal is to evaluate how logical optimizations impact both the prompt size and the number of iterations required to answer a query. For every query in the Internal Knowledge (IK) datasets, this task executes three distinct configurations of the system, all fixed to use the *Table-Scan* physical strategy.

The first configuration is **GALOIS $\emptyset$ (No-Push)**, which serves as the unoptimized baseline where no conditions are pushed to the LLM. The second is **GALOIS A (Push-All)**, an aggressive optimization that pushes all **WHERE** conditions into the prompt. The third is **GALOIS F**, the confidence-based approach that selectively pushes only high-confidence conditions.

For each run, the script logs the total number of input tokens, the number of iterations performed, and whether the execution converged within the maximum limit of 10 iterations.

The results from this replication, conducted using `llama-3.3-70b-instruct`, align closely with the findings of the original paper, confirming the significant efficiency gains provided by the optimizations. The baseline **GALOIS $\emptyset$** proves to be the most resource-intensive approach in terms of token consumption and execution time. Furthermore, this unoptimised strategy exhibits stability issues, failing to converge for 11 queries within the iteration limit. Conversely, applying logical optimizations drastically improves performance. **GALOIS A** reduces the average token count and the iterations, successfully eliminating all convergence failures. **GALOIS F** achieves the best overall efficiency, further lowering the token usage and the average iterations.

These findings demonstrate that pushing filter conditions down to the LLM not only reduces the volume of data processed but also creates a more robust and faster extraction process, preventing the model from getting stuck in long retrieval loops.

Table 19: Prompt size and iterations with GALOIS optimizations.

Method	Avg Input Tokens	Avg Iterations	Failures
GALOIS $\emptyset$ (No-Push)	1301	5.05	11
GALOIS A (Push-All)	1100	3.31	0
GALOIS F (Confident)	1090	3.26	0

Note: Results aggregated over 77 queries from IK datasets.

9.7 Experiment 7: Threshold Setup

This experiment focuses on calibrating the confidence threshold parameter, denoted as τ , which governs the GALOIS physical optimizer. The system uses this threshold to dynamically choose between two physical operators: *Table-Scan* (retrieving full tuples with broader context) and *Key-Scan* (iterative retrieval using Chain-of-Thought reasoning).

The selection logic dictates that if the estimated confidence of the model for the query exceeds τ , GALOIS employs the *Key-Scan* operator; conversely, if the confidence falls below τ , the system reverts to the *Table-Scan* strategy. To determine the optimal τ , we executed the system on the GEO-TEST dataset (used as a calibration set to avoid overfitting on the main benchmarks) while varying τ from 1.0 (forcing *Table-Scan*) down to 0.0 (forcing *Key-Scan*). We evaluated three models: `llama-3.3-70b-instruct`, `llama-3.1-8b-instruct`, and `gpt-4o-mini`.

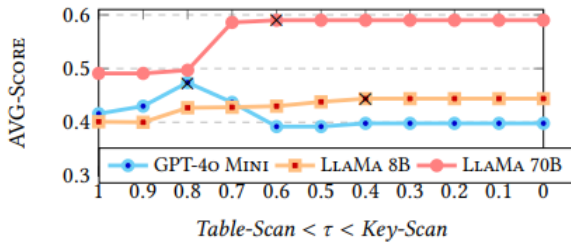


Figure 5: Original Paper Results

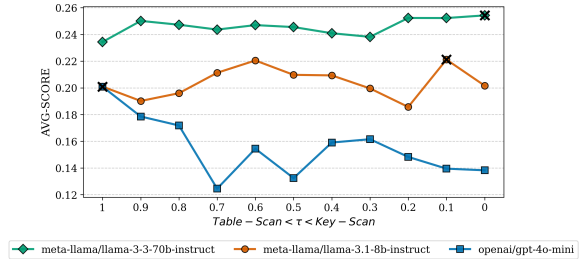


Figure 6: Our Reproduction Results

9.7.1 Analysis of Calibration Results

The results of our calibration runs, depicted in Figure 6 (right), present a distinct behavior compared to the findings in the original GALOIS paper (left).

Performance Discrepancy First, consistent with Experiment 3, the absolute performance in our runs is significantly lower. The original paper reports peak AVG-Scores between 0.4 and 0.6. In contrast, our best-performing model (`llama-3.3-70b`) peaks at approximately 0.25, and `gpt-4o-mini` peaks at 0.20.

Model-Specific Behavior and Optimal Thresholds The calibration curves reveal divergent optimal strategies for the different model sizes.

For **GPT-4o mini**, the curve demonstrates a negative correlation between *Key-Scan* usage and quality. The highest score is achieved at $\tau = 1$ (pure *Table-Scan*), and performance degrades as τ decreases. This contrasts with the paper, which identified an

optimal $\tau = 0.8$, suggesting that the iterative *Key-Scan* operator in our implementation is detrimental for this model, likely due to context fragmentation or the "zero recall" issue identified in the previous study.

Conversely, **LLaMA 3.3 70B** exhibits the opposite behavior. Performance gradually improves as we lower the threshold, peaking at $\tau = 0.1$ and $\tau = 0$ (pure *Key-Scan*). While the original paper found an optimal $\tau = 0.6$, our data suggests that for the largest model, the *Table-Scan* is insufficient, and the model relies entirely on the granular, iterative retrieval of *Key-Scan* to achieve its maximum accuracy.

Finally, the behavior of **LLaMA 3.1 8B** is noisy, with local maxima at both $\tau = 1$ and $\tau = 0.1$. This inconsistency suggests that the 8B model struggles to maintain stability in either mode, making it sensitive to the specific prompt structure rather than the logical operator choice.

In conclusion, our calibration suggests that the dynamic switching mechanism (using intermediate τ values) is less effective in our current reproduction than in the original study. The models tend to prefer binary extremes: the smaller model requires the context of *Table-Scan*, while the larger model requires the reasoning steps of *Key-Scan*.

9.8 Experiment 8: Errors w.r.t. the optimal plan

Objective and Methodology This experiment aims to quantify the efficacy of the **Logical Planner** (the component deciding which strategy to adopt) by measuring the "Optimality Gap". The Optimality Gap represents the performance difference between the automatic choice made by $GALOIS_F$ and the theoretical optimal choice ("Oracle") that the system could have made if it had selected the best possible strategy for every single query.

For every query present in the target datasets (GEO, WORLD, MOVIES), the runner executes the configurations in "brute-force" mode to determine the upper bound of performance. We define the Oracle score S_{opt} for each query q as the maximum score obtainable among the available strategies:

$$S_{opt}(q) = \max(S_{GALOIS_A}(q), S_{GALOIS_WO}(q), S_{GALOIS_F}(q))$$

The Optimality Gap is calculated as the average difference between the Oracle and the adaptive variant:

$$Gap = \frac{1}{N} \sum_{q \in Q} (S_{opt}(q) - S_{GALOIS_F}(q))$$

A gap close to 0 indicates that the automatic selector is choosing the best available strategy almost every time. A high gap suggests that, although a winning strategy exists for that query (e.g., *Key-Scan*), the system erroneously opted for a sub-optimal one.

The results presented in Figure 7 report the AVG-SCORE aggregated across all queries in the benchmark, and we also provide a breakdown of the AVG-SCORE for each individual dataset to highlight domain-specific performance.

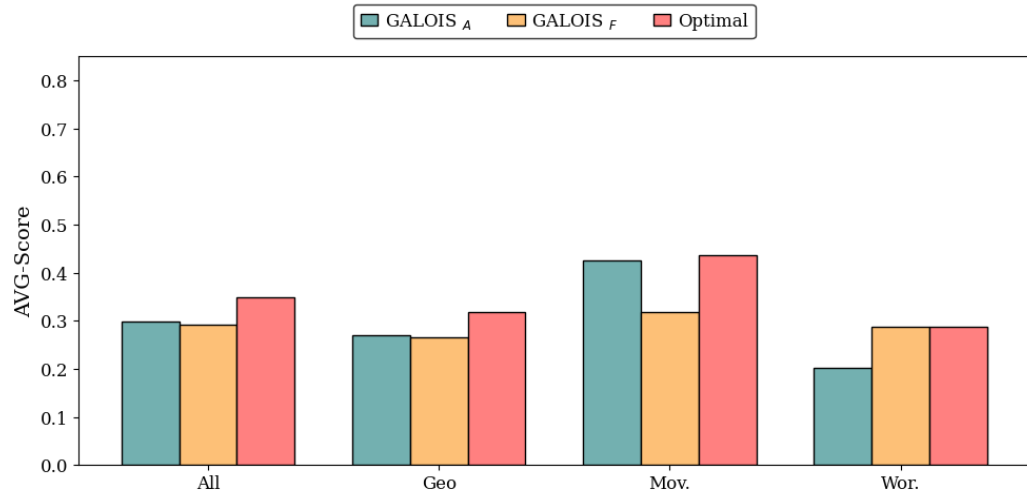


Figure 7: AVG-Score Comparison for Galois A and Galois F against the optimal plans across all datasets.

10 Implementation Challenges & Future Work

The development of the GALOIS Execution Engine presented several non-trivial implementation and performance challenges, particularly concerning the **Key-Scan** operator and database resource management.

10.1 Performance Deficiencies of the Key-Scan Operator

The most significant performance challenge was encountered during the development and testing of the **Key-Scan** physical operator, where initial evaluations demonstrated **poor retrieval effectiveness** under the GALOIS_F (Push-Confident) and GALOIS_{WO} (No-Push) logical plans.

The Locality-of-Reference Problem The poor performance stems from a fundamental design aspect of the **Key-Scan**’s two-phase strategy: the iterative Phase 1 is designed solely to retrieve unique primary key values ($\mathcal{K}_{all}$) without applying the initial query’s non-key predicates.

For instance, consider querying the `venezuela_presidents` table with `president_name` as the primary key. In Phase 1, the LLM is prompted to retrieve *all* keys globally available in the training data (e.g., all world presidents). The actual filter condition (e.g., `country = 'Venezuela'`) is applied only in the post-retrieval Phase 2 (the Tail Request). If the LLM’s initial response set $\mathcal{K}_{all}$ lacks sufficient locality (i.e., fails to yield keys related to 'Venezuela'), the final result set, post-query filtering, will be empty ($\mathcal{R}_{final} = \emptyset$). This retrieval inefficiency was common across several datasets used in the GALOIS_F and GALOIS_{WO} contexts.

10.2 Database Resource Management for DuckDB

A critical architectural challenge involved managing the concurrent access and connection lifecycle for the DuckDB backend. The execution pipeline requires numerous connection instantiations across different classes for tasks such as retrieving column data types, inspecting the database schema, and executing final queries.

To address the potential for file locks and resource contention on the underlying `.duckdb` instance, the resource management for the DuckDB backend adheres to a strict **acquisition-release pattern**. Database connections are instantiated solely within the confined scope of the active schema retrieval or query execution task and are immediately released within `finally` blocks upon completion. This explicit lifecycle management guarantees that the transition between the schema inspection phase and the subsequent in-memory virtualization phase proceeds without contention, maintaining system stability.

10.3 Addressing Sub-Optimal Prompt Structure

The core functionality of the **Table-Scan** and **Key-Scan** iterative models has been established; however, current system performance indicates that the **structure and semantic design of the prompts** generated by the existing GALOIS framework are sub-optimal. These prompts fail to consistently elicit high-fidelity, complete structured output from the LLMs.

This observation points to a critical area for future work: systematically improving the retrieval effectiveness by focusing on prompt engineering. Future efforts will concentrate on refining the instructions, contextualization ($\mathcal{H}$ and $\mathcal{P}_{pushdown}$ injection), and required output formats (e.g., explicit JSON schemas) to ensure the LLM provides more precise and comprehensive results. The hypothesis is that **optimized prompt semantics** will directly lead to a reduction in the number of required iterations, lower token consumption, and ultimately, higher data coverage.

11 Contributions

Alessio Demo (Badge number: 2142885)

- **First Deadline:** Developed function database connection and instance creation of DuckDB database manager for each dataset using the ingest files (.duckdb files). Developed function for retrieving the SQL queries from .sql files, processing the queries and executing them on DuckDB database instance and finally store it in a json file.
- **Second Deadline:** Implemented the Baseline NL function for interacting with the LLM with a backoff in case of gemini returns a free-plan limitation exception, save the results in FULL JSON format and next using it in the main script. Unit tests for the baseline_tools script using pytest. RAG task implementation using external libraries for creating the embeddings and evaluate the similarity between them in relation to a given prompt. Developed scripts for getting the db schema from DuckDB database for each dataset and the SQL query parsing, that will be used in the GALOIS implementation in the next tasks.
- **ThirdDeadline:** Deployed the query parser that translates SQL queries to an abstract syntax tree that makes it more easy to manipulate. Developed the Galois class that implements the logic handling the choice of physical and logical optimization techniques, also implemented the class GaloisEstimator that interacts with LLM for confidence rates involved in the process. Furthermore builded up the post-processing process that handles the execution of original query on the data frame containing the LLM response, and JOIN operations between multiple tables. Added costs optimization logic that asks the LLM to provide data only for the attributes that are involved in each query, instead of asking the entire set of attributes that make up the table. Developed Experiment 4, Experiment 5 and Experiment 8.

Writing the reports sections reporting the complete pipeline followed for developing the stuff above.

Francesco Pivotto (Badge number: 2158296)

- **First Deadline:** Implemented a modular project structure by relocating core logic to `src/` and consolidating configuration and database management logic within dedicated utility modules (`config/loaders.py`). Established the framework for multi-provider support by adding initial integration for the Grok LLM, including dedicated configuration points (`.env`, `config.yaml`, `loaders.py`).
- **Second Deadline:** Completed initial implementation of the SQL baseline, including the core logic for connecting to DuckDB and structuring query execution via the Watsonx provider. Fixed critical issues in environment variable loading, ensuring robust and standardized retrieval of secrets and settings via the central configuration object (using `loaders.py`). Modularized the `sql_baseline` module, incorporating the LCEL Pattern to prepare for dynamic LLM selection. Centralized general utility functions by moving helpers into `baseline_tools`. Introduced the dedicated `llm_wrapper` class to handle all LLM connection logic, API parameter management, and calls.
- **Third Deadline:** Implemented the core Galois Execution Engine (`GaloisExecutor`) and integrated its iterative physical operators (`Table-Scan` and `Key-Scan`). Decoupled the entire schema management architecture into modular components

(`base_schema_manager.py`, `schema_factory.py`).

Added logic to the Galois executor module to reproduce experiments 3 and 7 from the Galois paper.

Giorgia Amato (Badge number: 2159999)

- **First Deadline:** Contributed to the project initialization by setting up the repository structure, environment configuration, and shared utility scripts. Added support for `EXPLAIN` and `EXPLAIN ANALYZE` with JSON serialization, implemented automatic JSON generation for query outputs, and improved logging, debugging, and testing utilities.
- **Second Deadline:** Developed the Gemini-based SQL baseline, including IK/MC query handling, schema and raw-data injection into prompts, structured JSON parsing via Pydantic models, and robust error/timeout management. Refactored and extended unit tests for baseline tools and added new tests validating LLM interaction and Gemini-based SQL execution.
- **Third Deadline:** Implemented the iterative extensions for both SQL and NL baselines, introducing configurable execution via a `max_iter` parameter, tuple deduplication across iterations, and early-stopping conditions when no new tuples are returned. Standardized the aggregation and reporting of token usage and latency metrics for iterative executions, ensuring full compatibility with the existing JSON output schema.

Implemented the **GALOIS-WO Table-Scan** pipeline, including the `GaloisExecutor`, the iterative execution loop (initial and iterative prompts), stopping criteria, and robust JSON parsing and deduplication logic to guarantee unique tuple collection. Developed the `GaloisWOSchemaManager` for DuckDB schema inspection, table and attribute discovery, table-name resolution, key extraction, and automatic JSON schema example generation.

Designed and implemented the `demo_process_single_query` debugging utility, enabling step-by-step inspection of GALOIS execution variants (WO, A, S, F), including parser output, execution plans, intermediate debug information, statistics, and partial result inspection.

Fully implemented **Experiment 2 (Physical and Logical Optimization)**, including the experiment runner, evaluation pipeline, metric computation, result aggregation, and reproducible summary generation, ensuring strict alignment with the experimental protocol described in the GALOIS paper.

Initiated the implementation of **Experiment 6 (Evolution of Prompt Size)**, defining the experimental logic, metrics, and execution structure for measuring prompt growth across iterations in GALOIS; the completion and extension of this experiment were subsequently carried out by another team member.

Contributed to the writing and refinement of the report sections describing the iterative baselines, the GALOIS-WO Table-Scan algorithm, Experiment 2, and the corresponding numerical results.